

Laboratory 6

Variant 4

Group 19

Mathis Morra-Fischer & Nolan Capomaggio

Task Description

The objective of this lab is to implement the Q-Learning algorithm to solve a reinforcement learning problem from the Gymnasium library. The assigned variant is Variant 4, which uses the Taxi-v3 environment.

The agent learns a policy by interacting with the environment. At each step, it chooses an action, observes the next state and reward, and updates a Q-table using the Q-Learning update rule. The goal of the experiments is to study the influence of different hyperparameters, such as the learning rate, discount factor, exploration rate, and number of training episodes.

Environment Description

The selected environment is Taxi-v3. The goal is to control a taxi that must pick up a passenger at one location and drop the passenger off at the correct destination.

The environment has a discrete state space and a discrete action space. Therefore, no quantization is required. The Q-table can directly represent all state-action values.

The action space contains 6 possible actions:

```
0: move south
1: move north
2: move east
3: move west
4: pick up passenger
5: drop off passenger
```

The reward system is:

```
+20 for a successful drop-off
-1 for each time step
-10 for illegal pickup or drop-off actions
```

An episode terminates when the passenger is successfully dropped off. An episode can also be truncated if the maximum number of steps is reached, here it is set to 200 steps per episode.

Q-Learning Implementation

The Q-table is initialized with zeros and has one row for each state and one column for each action. Since Taxi-v3 has 500 discrete states and 6 discrete actions, the Q-table has size 500×6 .

The Q-values are updated using the standard Q-Learning rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where α is the learning rate and γ is the discount factor. If the next state is terminal, the target is only the immediate reward, because no future reward can be obtained from a terminal state.

The agent uses an epsilon-greedy policy during training. With probability epsilon, it selects a random action to explore. Otherwise, it selects the action with the highest Q-value for the current state.

The exploration rate starts at 100% and decreases linearly toward a minimum value:

$$\epsilon_{step} = (100 - \epsilon_{min}) / \text{train_iter}$$

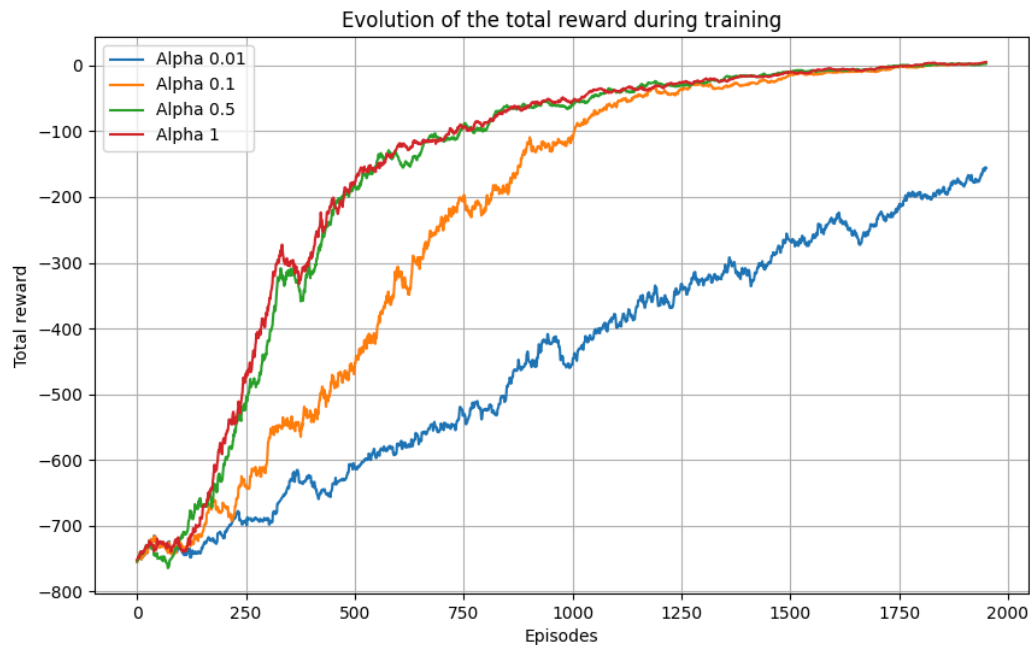
For episode t , with t starting from 1:

$$\epsilon(t) = 100 - t \times \epsilon_{step}$$

In the implementation, epsilon is represented as a percentage. It is divided by 100 when used as a probability. A fixed seed of 42 is used to make the experiments reproducible.

Experiments and Results

Learning Rate Experiment



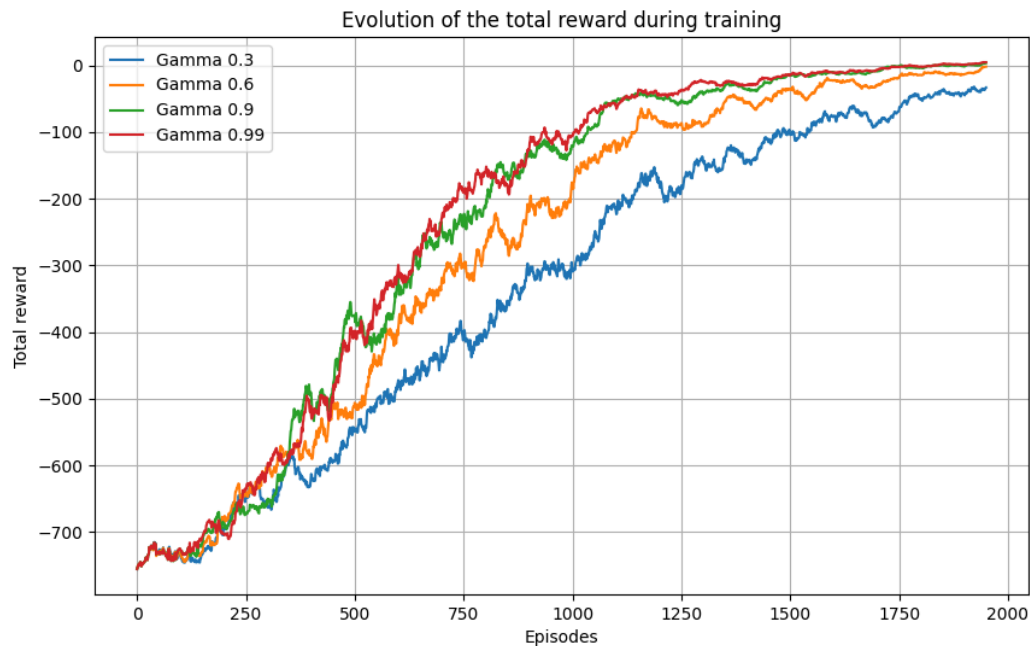
The tested learning rates were $\alpha = 0.01$, 0.1 , 0.5 , and 1 . The other hyperparameters were fixed to $\gamma = 0.95$, $\text{min_eps} = 5\%$, and 2000 training episodes.

The results show that $\alpha = 0.01$ learns too slowly. Its reward remains much lower than the others after 2000 episodes because each update only slightly modifies the Q-values.

The values $\alpha = 0.5$ and $\alpha = 1$ converge faster in this experiment. However, $\alpha = 1$ is aggressive because the old Q-value is completely replaced by the new target at each update. This can amplify noisy updates and may be less stable in more complex environments.

The value $\alpha = 0.1$ learns more slowly but provides a conservative and stable baseline. It was selected for the final configuration because it still reaches a 100% success rate after sufficient training.

Discount Factor Experiment



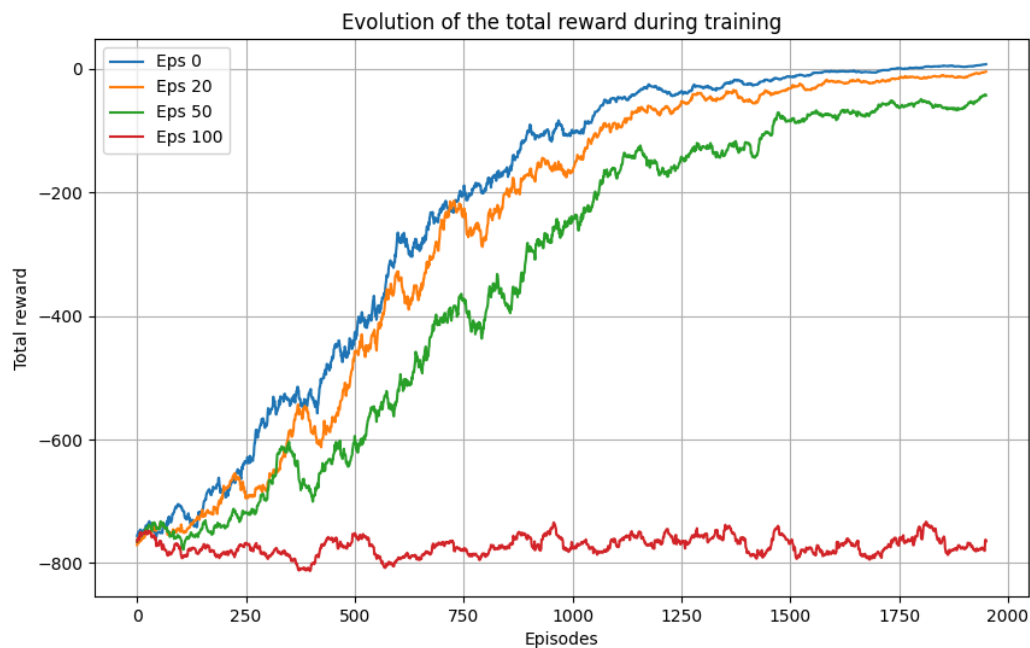
In this experiment, the discount factor γ was varied. The tested values were $\gamma = 0.3, 0.6, 0.9$, and 0.99 . The learning rate was fixed to $\alpha = 0.1$, the minimum exploration rate was fixed to 5%, and the number of training episodes was 2000.

The discount factor controls how much the agent values future rewards. A low value of γ makes the agent focus mainly on immediate rewards, while a high value encourages the agent to consider long-term consequences.

The results show that $\gamma = 0.3$ performs worse than the higher values. This is expected in Taxi-v3 because the main positive reward, +20, is received only after successfully dropping off the passenger. This reward is delayed, so the agent needs to value future rewards to learn an efficient policy.

The curves for $\gamma = 0.9$ and $\gamma = 0.99$ perform the best. They improve faster and reach higher total rewards. This confirms that a high discount factor is useful in this environment because the task requires planning several actions ahead.

Exploration Rate Experiment



In this experiment, the minimum exploration rate was varied. The initial exploration rate was 100% and decreased linearly during training until it reached the selected minimum value. The tested minimum exploration rates were 0%, 20%, 50%, and 100%. The learning rate was fixed to $\alpha = 0.1$, the discount factor was fixed to $\gamma = 0.95$, and the number of training episodes was 2000.

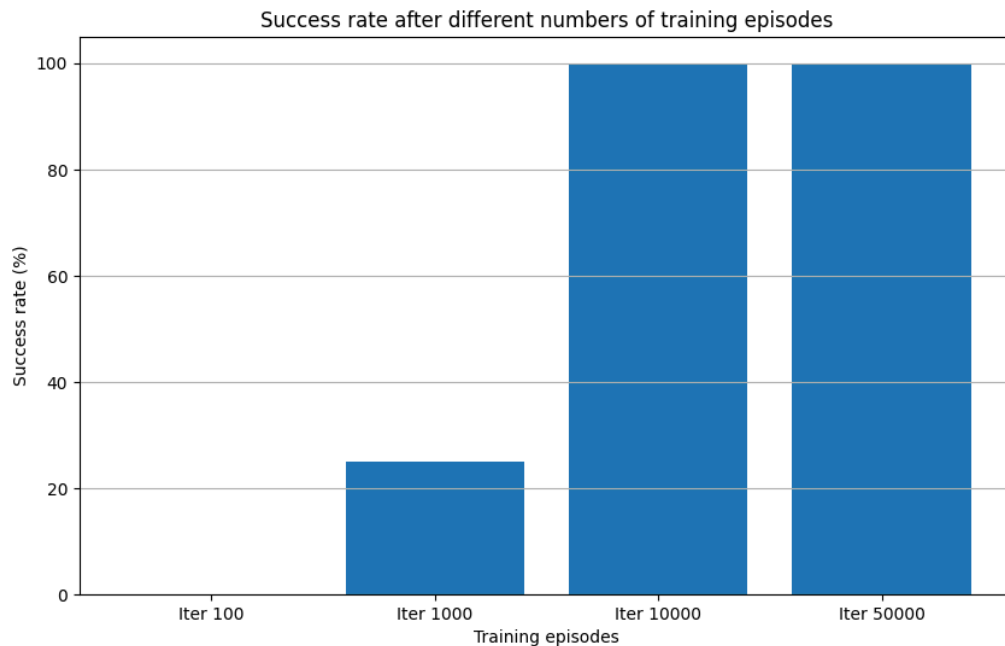
The results show that a minimum exploration rate of 0% gives the best final reward. This is because the agent explores a lot at the beginning, but becomes greedy near the end of training and uses the learned Q-values more consistently.

A minimum exploration rate of 20% still gives good results, but remains slightly worse because the agent continues to choose random actions even after it has learned a useful policy.

A minimum exploration rate of 50% performs worse because the agent still behaves randomly in half of the decisions at the end of training. This prevents the total reward from becoming as high as with lower exploration rates.

The minimum exploration rate of 100% performs very badly. In this case, the agent always acts randomly and never exploits the learned Q-table. As a result, the total reward remains very negative and the policy does not improve.

Number of Training Episodes Experiment



In this experiment, the number of training episodes was varied. The tested values were 100, 1000, 10000, and 50000 episodes. The other hyperparameters were fixed to $\alpha = 0.1$, $\gamma = 0.95$, and minimum exploration rate = 5%. After training, each policy was evaluated over 100 test episodes without exploration.

Training episodes	Success rate	Average reward	Average steps
100	0.00%	-218.00	200.00
1000	25.00%	-147.49	152.74
10000	100.00%	7.34	13.66
50000	100.00%	7.34	13.66

The results show that the number of training episodes has a strong impact on the final policy. With only 100 episodes, the agent does not learn a successful policy. The success rate is 0%, the average reward is very negative, and the average number of steps reaches the maximum value of 200. This means that the agent usually fails to complete the task within the allowed number of steps.

With 1000 episodes, the agent starts to learn useful behavior. The success rate increases to 25%, and the average number of steps decreases. However, the policy is still unreliable and the average reward remains strongly negative.

With 10000 episodes, the policy reaches a 100% success rate. The average reward becomes positive and the average number of steps decreases to about 13.7. This indicates that the agent has learned an efficient policy.

Training for 50000 episodes does not improve the results compared to 10000 episodes. Both configurations obtain the same success rate, average reward, and average number of steps. This suggests that the policy has already converged around 10000 episodes, and additional training does not bring visible benefits.

Final Evaluation

The selected final configuration was:

```
learning rate  $\alpha$  = 0.1  
discount factor  $\gamma$  = 0.95  
minimum exploration rate = 5%  
training episodes = 10000  
maximum steps per episode = 200  
evaluation episodes = 100  
seed = 42
```

The final policy achieved the following results:

```
Success rate: 100.0%  
Average reward: 7.34  
Best reward: 15.00  
Worst reward: 3.00  
Average number of steps: 13.7
```

These results show that the learned policy solves the Taxi-v3 environment reliably. The agent successfully completes all evaluation episodes and does so in a small number of steps.

Conclusion

This lab implemented the Q-Learning algorithm on the Taxi-v3 environment. Since the environment has discrete states and actions, a tabular Q-table could be used directly without quantization.

The experiments showed that the learning rate, discount factor, exploration rate, and number of training episodes strongly affect learning. A very small learning rate slows down convergence, while a high discount factor is useful because the main positive reward is delayed until the passenger is successfully dropped off. The exploration experiment showed that the agent needs exploration at the beginning, but must reduce exploration later to exploit the learned policy.

The final configuration reached a 100% success rate over 100 evaluation episodes, with an average reward of 7.34 and an average number of 13.66 steps.

However, this approach is suitable mainly because Taxi-v3 has a small discrete state space. For larger or continuous environments, a tabular Q-table would become impractical, and methods based on function approximation, such as Deep Q-Networks, would be more appropriate.